

Compromission du RSA avec l'algorithme de Shor

H. B.

Mai 2024

Abstract

La sécurité des communications numériques modernes repose en grande partie sur des problèmes mathématiques réputés difficiles pour l'informatique classique, notamment la factorisation de grands nombres premiers utilisée par l'algorithme RSA. L'émergence de l'informatique quantique remet en cause cette hypothèse fondamentale. Grâce aux propriétés de superposition et d'intrication des qubits, l'algorithme de Shor permet théoriquement de factoriser un entier en temps polynomial, compromettant ainsi les schémas cryptographiques actuels fondés sur la difficulté computationnelle.

Cet article présente les bases nécessaires à la compréhension de cette rupture technologique. Après un rappel des fondements de l'informatique classique et de la logique booléenne, une introduction progressive aux principes de l'informatique quantique est proposée. Les notions de qubit, de superposition, d'opérateurs unitaires et de transformée de Fourier quantique sont introduites afin d'expliquer le fonctionnement de l'algorithme de Shor et son impact potentiel sur la sécurité de RSA.

Introduction

Depuis plusieurs décennies, la sécurité des communications numériques repose sur une hypothèse simple mais fondamentale, à savoir que certains problèmes mathématiques sont faciles à vérifier mais extrêmement difficiles à résoudre. La factorisation d'un grand entier en nombres premiers appartient à cette catégorie. Cette asymétrie constitue le socle du chiffrement RSA, largement utilisé pour protéger les échanges sur Internet, les transactions bancaires et les infrastructures critiques.

Cependant, cette sécurité n'est pas absolue. Elle dépend directement des limites de l'informatique classique. Or, l'émergence de l'informatique quantique introduit un changement de paradigme. En exploitant les lois de la physique quantique, un ordinateur quantique ne manipule plus des bits binaires déterministes, mais des qubits capables de superposition et d'intrication. Ces propriétés permettent d'envisager des accélérations algorithmiques spectaculaires pour certains problèmes spécifiques.

En 1994, Peter Shor propose un algorithme quantique capable de factoriser un entier en temps polynomial. Cette découverte transforme l'informatique quantique d'une curiosité théorique en enjeu stratégique mondial. Si un ordinateur quantique suffisamment puissant venait à être construit, il pourrait compromettre certains systèmes cryptographiques actuellement déployés.

Afin de comprendre pourquoi l'algorithme de Shor représente une menace pour RSA, il est nécessaire d'examiner les fondements de l'informatique classique et quantique. Cet article propose une progres-

sion pédagogique partant des transistors et de la logique booléenne jusqu'à la transformée de Fourier quantique, outil central de la recherche de période dans l'algorithme de Shor.

L'informatique classique

L'informatique classique n'est possible que par la miniaturisation du transistor. Les microprocesseurs sont constitués de plusieurs centaines de millions de ces briques élémentaires (elles-mêmes constituées d'une superposition de matériaux semi-conducteur). Les microprocesseurs actuels utilisent la technologie **CMOS** (*Complementary Metal Oxide Semiconductor*) qui fait appel à la fois à des transistors de type **p** et des transistors de type **n**. Un transistor possède 3 broches nommées collecteur, base et émetteur et est constitué de trois couches **p-n-p** (type **p**) ou **n-p-n** (type **n**). Le dopage **n/p** du silicium consiste en l'ajout d'impuretés facilitant soit le déplacement des électrons, soit le déplacement des trous. Un transistor se comporte comme un interrupteur entre l'émetteur et le collecteur commandé par la valeur de la tension sur la base.

Prenons l'exemple d'un transistor de type **n**. Il est constitué de deux jonctions: une jonction **n-p** et une jonction **p-n** qui se comportent alors comme des diodes en opposition. Entre ces deux jonctions se trouve la base. D'après le principe de fonctionnement d'une diode (le courant ne passe que dans un sens), il s'avère qu'en l'absence de tension sur la base, aucun courant ne peut circuler entre l'émetteur et le collecteur. Lorsqu'une tension positive est appliquée sur la base, les électrons (chargés négativement) vont

Table 1: Table de vérité d'une porte NOT

NOT	
Entrée	Sortie
1	0
0	1

Table 2: Table de vérité d'une porte AND

AND		
Entrée 1	Entrée 2	Sortie
1	1	1
1	0	0
0	1	0
0	0	0

alors s'accumuler près de la base, modifiant ainsi localement le dopage, laissant donc passer le courant entre l'émetteur et le collecteur.

Le lecteur peut se demander quel est le lien entre le transistor et l'informatique telle qu'il la connaît. L'objectif de l'informatique est principalement d'effectuer des calculs. Un microprocesseur n'est rien d'autre qu'une machine à calculer sophistiquée. En assemblant des transistors d'une certaine manière, il est possible d'effectuer des calculs. On parle d'algèbre de Boole. Cette algèbre de Boole permet de déterminer la valeur d'un signal de sortie en fonction d'un ou plusieurs signaux d'entrée dans une "boîte noire". Pour inverser la valeur d'un signal et passer ainsi d'un bit qui vaut 1 à un bit qui vaut 0 (ou inversement), il faut que cette "boîte noire" soit un inverseur. L'inverseur est constitué de deux transistors, l'un de type **p** et l'autre de type **n**. L'inverseur est donc la "boîte noire" et elle s'appelle une porte logique **NOT** dont la table de vérité est présentée dans le tableau 1.

Il existe d'autres portes logiques permettant d'effectuer d'autres calculs, comme la porte **AND** qui prend 2 valeurs en entrée et renvoi une valeur en sortie. Elle est définie par la table de vérité présentée dans le tableau 2.

Les portes **OR**, **NOT AND (NAND)**, **NOT OR (NOR)** et **XOR** sont présentées dans les tableaux 3, 4, 5 et 6 respectivement.

Afin d'introduire les mathématiques nécessaires à la compréhension de l'informatique quantique, tentons d'exprimer la logique booléenne présentée ci-dessus en termes de calcul matriciel. Par définition, l'état d'un bit est soit 0, soit 1. L'ensemble des états possibles peut alors être représenté sous forme vectoriel. L'état 0 et l'état 1 d'un bit sont alors représentés par les relations 1.

Table 3: Table de vérité d'une porte OR

OR		
Entrée 1	Entrée 2	Sortie
1	1	1
1	0	1
0	1	1
0	0	0

Table 4: Table de vérité d'une porte NAND

NAND		
Entrée 1	Entrée 2	Sortie
1	1	0
1	0	1
0	1	1
0	0	1

Table 5: Table de vérité d'une porte NOR

NOR		
Entrée 1	Entrée 2	Sortie
1	1	0
1	0	0
0	1	0
0	0	1

Table 6: Table de vérité d'une porte XOR

XOR		
Entrée 1	Entrée 2	Sortie
1	1	0
1	0	1
0	1	1
0	0	0

$$\begin{aligned} |0\rangle &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ |1\rangle &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{aligned} \quad (1)$$

La porte **NOT** peut également être représentée sous forme matricielle comme cela est présenté sur la relation 2.

$$NOT = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (2)$$

Ainsi, appliquer un état d'entrée $|0\rangle$ à une porte **NOT** revient à effectuer le calcul suivant :

$$\begin{aligned} NOT |0\rangle &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 0*1 + 1*0 \\ 1*1 + 0*0 \end{pmatrix} \\ &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle \end{aligned} \quad (3)$$

On retrouve bien l'état $|1\rangle$ après avoir appliqué une porte **NOT** à un état $|0\rangle$. Les autres portes sont légèrement plus complexes. En effet, elles prennent deux états en entrée et fournissent un état en sortie. Il faut donc trouver un moyen de représenter vectoriellement ces deux états d'entrée dans un seul vecteur. La solution à ce problème s'appelle le produit tensoriel défini par le symbole $\otimes$. Le produit tensoriel d'un état $|0\rangle$ par un état $|0\rangle$ se note alors :

$$\begin{aligned} |0\rangle \otimes |0\rangle &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 1*1 \\ 1*0 \\ 0*1 \\ 0*0 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |00\rangle \end{aligned} \quad (4)$$

La porte **AND** peut alors se traduire sous forme matricielle comme suit :

$$AND = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (5)$$

Le produit tensoriel de l'état $|0\rangle$ et de l'état $|0\rangle$ dans une porte **AND** donne donc le résultat suivant :

$$\begin{aligned} AND |00\rangle &= \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 1*1 + 0*0 + 0*0 + 0*0 \\ 0*0 + 0*0 + 0*0 + 1*0 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle \end{aligned} \quad (6)$$

Deux états $|0\rangle$ en entrée d'une porte **AND** donne bien un état de sortie $|0\rangle$ comme cela est présenté dans la table de vérité 2. Le lecteur peut vérifier les autres éléments de la table de vérité pour s'assurer de la pertinence de cette matrice **AND**. Suivant le même principe, les matrices **OR**, **NAND**, **NOR** et **XOR** s'écrivent alors :

$$\begin{aligned} OR &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix} \\ NAND &= \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \\ NOR &= \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} \\ XOR &= \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} \end{aligned} \quad (7)$$

Le lien entre le transistor et les bases de l'informatique classique moderne a alors été établi. Les briques élémentaires de l'informatique classique ont été développées sous forme matricielle, et les différents états que peut prendre un bit sous forme vectoriel. La notion de produit tensoriel a également été abordée. Les possibilités de calcul en termes de logique sont alors définies par les tables de vérité et par la non-superposition d'état. Cette informatique peut être qualifiée de déterministe car il est possible de déterminer l'état de chaque bit à chaque instant. A contrario, comme cela sera montré dans la suite de cet article, l'informatique quantique sera qualifiée de probabiliste car l'état de chaque qubit n'est alors qu'une probabilité dont l'état ne peut être fixé que lors d'une mesure.

Afin de faire le lien avec l'informatique quantique, une dernière porte sera introduite. Il s'agit de la porte **CNOT** (*Controlled NOT*). Cette porte prend deux états d'entrée et fournit deux états de sortie. Elle agit comme une porte **NOT** sur le deuxième état si et seulement si le premier état (de contrôle) est $|1\rangle$. Sa matrice est donc :

$$CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (8)$$

Ce qui donne les résultats suivants (que le lecteur pourra vérifier) :

$$\begin{aligned} CNOT |00\rangle &= |00\rangle \\ CNOT |01\rangle &= |01\rangle \\ CNOT |10\rangle &= |11\rangle \\ CNOT |11\rangle &= |10\rangle \end{aligned} \quad (9)$$

L'informatique quantique

L'informatique quantique tente d'utiliser les propriétés d'enchevêtrement et de superposition d'états. Les bits ne sont donc plus à considérer de manière classique, mais de manière quantique : on parle de qubits (ou q-bits). Afin de comprendre les développements à suivre, des notions de physique quantique doivent être présentées. L'article étant essentiellement destiné à des non-physiciens, des raccourcis parfois grossiers, seront empruntés à grandes enjambées. En physique, une particule peut soit être décrite comme un corpuscule, soit comme une onde. À chaque particule est alors associée une fonction d'onde ($\psi(r, t)$) qui décrit son état en fonction de sa position et du temps. $\psi(r, t)$ est interprétée comme une amplitude de probabilité de présence. Les états possibles d'un électron sont donc définis par sa fonction d'onde et il est alors possible de définir un vecteur associé à l'ensemble des états possibles qui se notera $|\psi\rangle$. L'évolution temporelle d'un électron est régie par l'équation de Shrödinger comme suit :

$$i\hbar \frac{d}{dt} |\psi\rangle = H(t) |\psi\rangle \quad (10)$$

La notion d'état est bien claire dans le contexte de l'informatique classique, en revanche, elle apparaît légèrement plus vague dans le cadre de l'informatique quantique. Il existe plusieurs manières de fabriquer un qubit. Il peut s'agir du spin des électrons, des niveaux d'énergie dans un atome, de la polarisation des photons, etc. La notion d'ensemble d'états étant définie sous forme de vecteurs (appelé kets dans le contexte de la physique quantique), il est possible d'établir la transformation de ceux-ci suite à l'opération d'une "porte logique" (un opérateur). L'application d'un opérateur H à un ket $|\psi\rangle$ se note de la façon suivante (ce qui rappelle l'application d'une porte à un/plusieurs bits) :

$$H |\psi\rangle = \lambda |\psi\rangle \quad (11)$$

Cette équation n'a en générale de solutions que lorsque λ prend des valeurs bien spécifiques (quantification), dite valeurs propres qui sont en générale complexes. On parle donc d'équation aux valeurs propres de l'opérateur linéaire H (l'ensemble de ces

valeurs propres s'appelle le spectre de H). Dans le cadre de l'informatique quantique on s'intéresse aux valeurs propres réelles qui n'existent que pour des opérateurs particuliers dit hermitiques. De la même manière qu'en informatique classique, on s'arrangera en informatique quantique pour que le qubit ait deux états possibles $|0\rangle$ et $|1\rangle$. Cependant, comme énoncé précédemment, la différence majeure avec les bits est que la superposition d'état est possible. C'est-à-dire qu'il est possible d'obtenir un mélange de l'état $|0\rangle$ avec l'état $|1\rangle$, ce qui se traduit comme :

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad (12)$$

Dans cette équation, α et β représentent les amplitudes de probabilité (valeur propre de l'équation aux valeurs propres) de trouver la particule dans l'état $|0\rangle$ ou $|1\rangle$ ou un mélange des deux (vecteurs propres de l'équation aux valeurs propres). De plus, en informatique quantique, il est possible d'avoir des états qui dépendent l'un de l'autre (enchevêtrement quantique). Ainsi la modification d'un état entraîne nécessairement la modification de l'autre état (preuve d'interception de l'information). Par définition, les amplitudes de probabilités doivent satisfaire $|\alpha|^2 + |\beta|^2 = 1$. A partir de l'équation aux valeurs propres 11, il est alors possible de déterminer des matrices hermitiennes (définition hors du cadre de cet article) qui satisferont le pré-requis d'avoir les deux vecteurs propres $|0\rangle$ et $|1\rangle$. Les différentes matrices seront alors la représentation des portes logiques quantiques. On retrouve évidemment les portes logiques classiques (de manière à ne pas perdre en fonctionnalités auxquelles il est possible d'en ajouter de nouvelle) comme par exemple la porte Hadamard qui est définie par l'opérateur suivant :

$$H_{Hadamard} = H_H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (13)$$

Appliquer cette porte quantique aux états propres $|0\rangle$ et $|1\rangle$ donne alors les résultats suivants :

$$\begin{aligned} H_H |0\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\ H_H |1\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \end{aligned} \quad (14)$$

Ce qui se traduit par un résultat important de l'informatique quantique, à savoir la superposition des états propres. Cette porte ne prend en entrée qu'un seul état propre, tout comme la porte NOT

vue en informatique classique (équation 2). Il est intéressant de regarder ce qu'il se passe lorsque l'on applique la porte Hadamard en parallèle à deux qubits $|0\rangle$. Le produit tensoriel de deux qubits $|0\rangle$ donne $|00\rangle$ et le produit tensoriel de deux opérateurs d'Hadamard est :

$$\begin{aligned} H_{2H} &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \end{aligned} \quad (15)$$

Ainsi, on obtient l'état propre suivant :

$$\begin{aligned} H_{2H} |00\rangle &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \\ &= \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \end{aligned} \quad (16)$$

Le résultat est fascinant car on se retrouve alors avec un état de sortie qui a une amplitude de probabilité d' $1/2$ pour chaque mélange d'état possible. Il y a donc 1 chance sur 4 (le carré de $1/2$) d'observer l'un des états (état qui ne sera fixé qu'après la mesure). De manière à maximiser l'enchevêtrement de deux qubits, on souhaiterait obtenir le résultat suivant :

$$X |00\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \quad (17)$$

avec X un opérateur à déterminer. Ce résultat est connu sous le nom d'état de Bell. L'opérateur X est une application parallèle de plusieurs opérateurs, tout d'abord un opérateur Hadamard avec un opérateur Identité (opérateur qui ne change pas la valeur d'un qubit) comme suit :

$$\begin{aligned} H_H \otimes I &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix} \end{aligned} \quad (18)$$

L'application à deux qubits $|0\rangle$ donne alors :

$$H_H \otimes I |00\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \quad (19)$$

Si on applique ensuite cette sortie à une porte **CNOT** quantique (qui pour rappel inverse le second qubit uniquement si le premier est $|1\rangle$) on obtient alors :

$$\begin{aligned} CNOT H_H \otimes I |00\rangle &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \\ &= \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) \end{aligned} \quad (20)$$

On obtient bien le résultat escompté qui est un enchevêtrement maximal des états propres $|0\rangle$ et $|1\rangle$. Le mélange d'état permet de calculer instantanément l'ensemble des possibilités d'un calcul donné. Sans rentrer dans les détails, il est important de mentionner qu'il existe de nombreuses portes logiques comme les portes de Pauli, des portes rotatrices de phase, de nombreuses portes de contrôle (plus avancé que la porte **CNOT**), des portes qui agissent sur plus de 2 qubits d'entrées comme les portes de Fredkin et de Toffoli. Les portes classiques déjà présentées sont également réalisable car l'objectif de l'informatique quantique est d'augmenter les possibilités et fonctionnalités, et non d'en perdre. Un des défis de l'informatique quantique est de réaliser une mesure correcte. En effet, il est possible d'obtenir un mélange d'état contenant l'ensemble des résultats possibles d'un calcul, cependant il est parfois nécessaire de récupérer la valeur de ces possibilités. Pour cela, il faut effectuer une mesure, ce qui réduit forcément l'ensemble des possibilités à un seul des vecteurs propres (celui qui est mesuré) détruisant alors les autres possibilités. On parle de destruction du paquet d'onde (ne pas oublier qu'un qubit est assimilé à une fonction d'onde). On peut ainsi stocker un nombre important d'information mais il n'est pas possible d'y accéder. Il faut donc manipuler les opérateurs de manière à obtenir l'information souhaitée, comme une période, une moyenne, etc.

L'algorithme de Shor

Depuis les débuts de l'informatique quantique, différents algorithmes ont été imaginés. On peut citer l'algorithme de Deutsch, celui de Grover, mais celui qui sera d'intérêt ici est l'algorithme de Shor. Mis en place au milieu des années 90, cet algorithme permet de trouver les facteurs premiers des très grands nombres. La factorisation en nombre premier est la base du chiffrement à clef publique/privée **RSA**

(Rivest-Shamir-Adleman) (et d'autres). L'algorithme de Shor est donc celui qui motive l'ensemble des grandes puissances étatiques à développer le secteur de l'informatique quantique.

Encore une fois, des notions de physique et notamment de traitement du signal sont nécessaires pour comprendre cette section. Afin de ne pas perdre le lecteur dans des développements mathématiques complexes, seules les formules essentielles seront présentées.

RSA est un algorithme à clef privée/publique qui fonctionne grâce à la difficulté de décomposer un nombre en ses facteurs premiers. Prenons deux grands nombres premiers et multiplions les. Le résultat est alors la clef publique qui sera transmise en clair aux différents destinataires avec lesquels on souhaite communiquer. Le destinataire utilise alors cette clef pour chiffrer un message. Tout le monde est capable de chiffrer un message avec la clef publique, mais seul le propriétaire de la clef privée peut le déchiffrer. En effet, lui seul connaît les 2 nombres premiers qui ont servi à fabriquer la clef publique. Prenons l'exemple suivant. Soit deux nombres premiers Q et R avec $Q = 3$ et $R = 5$, le produit P vaut alors 15 qui est la clef publique. La solidité de **RSA** tient du fait qu'il est extrêmement difficile de retrouver Q et R sachant P mais qu'il est extrêmement facile de vérifier que leur produit donne bien P . L'algorithme de Shor, est actuellement la méthode la plus efficace pour effectuer cette factorisation. Avec l'informatique classique, elle est de complexité $O(2^n)$, ce qui la rend pratiquement infaisable en temps court pour de très grandes nombres. En revanche, grâce au mélange d'état que procure l'informatique quantique, il devient possible de l'exécuter dans des temps acceptables.

L'algorithme de Shor est le suivant:

1. Choisir un nombre a aléatoire, par exemple $a = 7$.
2. Calculer la fonction $f(x) = a^x \bmod P$.
3. Trouver la périodicité, c'est-à-dire r tel que $f(x) = f(x+r) = \dots = f(x+kr)$.
4. Si r est impair, recommencer avec un autre a , sinon les nombres premiers sont donnés par $Q = \text{pgdc}(a^{r/2} - 1, P)$ et $R = \text{pgdc}(a^{r/2} + 1, P)$ (où $\text{pgdc}()$ est une fonction qui calcule le plus grand dénominateur commun).

Ainsi pour $P = 15$, prenons $a = 7$, on a alors les valeurs suivantes pour $f(x) = a^x \bmod P$:

- $7^0 \bmod 15 = 1$
- $7^1 \bmod 15 = 7$
- $7^2 \bmod 15 = 4$
- $7^3 \bmod 15 = 13$
- $7^4 \bmod 15 = 1$

- $7^5 \bmod 15 = 7$
- $7^6 \bmod 15 = 4$
- $7^7 \bmod 15 = 13$
- etc

Les valeurs 1, 7, 4, 13 se répètent avec une périodicité de 4. Ainsi $r = 4$. r est pair, on peut alors calculer Q et R de cette manière:

- $Q = \text{pgdc}(48, 15) = 5$
- $R = \text{pgdc}(50, 15) = 3$

On retrouve bien $Q = 5$ et $R = 3$. Il existe des algorithmes efficaces pour calculer le plus grand dénominateur commun, comme par exemple l'algorithme d'Euclide. Ainsi la grande difficulté de cet algorithme de Shor est la détermination de la périodicité r . Heureusement il existe un outil mathématique appelé la transformée de Fourier discrète qui permet de déterminer la périodicité d'un signal discret. La **DFT** (*Discret Fourier Transform*) se note comme sur la relation 21

$$\beta_{k < N} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N} \quad (21)$$

avec N le nombre d'éléments et i le nombre imaginaire avec $i^2 = -1$. La **DFT** pour l'ensemble de valeurs $x_j = \{1; 3\}$ est alors ($N = 2$):

$$\begin{aligned} \beta_0 &= \frac{1}{\sqrt{2}} \sum_{j=0}^1 x_j e^0 = \frac{4}{\sqrt{2}} \\ \beta_1 &= \frac{1}{\sqrt{2}} \sum_{j=0}^1 x_j e^{\pi i j} = -\frac{2}{\sqrt{2}} \end{aligned} \quad (22)$$

La transformée de Fourier permet de décrire un ensemble d'éléments en différentes fréquences et amplitudes le composant. Ainsi le résultat final est un spectre dont il convient de prendre les valeurs maximales en amplitudes afin de trouver les fréquences les plus importantes du signal et donc la périodicité r . Reprenons l'exemple dans lequel $P = 15$. L'ensemble sur lequel effectuer la **DFT** est donc $\{1; 7; 4; 13; 1; 7; 4; 13\}$. L'ensemble réel résultant de la transformée de Fourier discrète est alors $\{50; 0; 13,4; 0; 30; 0; 13,4; 0\}$. Regardons l'indice du pic avec l'amplitude la plus élevée sans compter la première valeur qui est un *offset*. La valeur 30 se situe alors à l'indice 4 ce qui correspond à la périodicité r qui a été trouvée manuellement. La **DFT** est alors facile à calculer, mais cela prend du temps. L'intérêt d'effectuer une **DFT** quantique (**QFT**) est alors d'effectuer l'ensemble des calculs en parallèle. La destruction du paquet

d'onde et donc la mesure permettra de déterminer la périodicité qui s'avérera être la variable la plus complexe à déterminer dans l'algorithme de Shor.

La **QFT** s'applique sur les valeurs propres, à savoir sur les amplitudes des états propres. Ainsi pour un état mélangé à 2 qubits dont la fonction d'onde est la suivante:

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle \quad (23)$$

les valeurs issues de la **QFT** sont alors:

$$\begin{aligned} \beta_0 &= \frac{1}{2}(\alpha_{00} + \alpha_{01} + \alpha_{10} + \alpha_{11}) \\ \beta_1 &= \frac{1}{2}(\alpha_{00} + \alpha_{01}i - \alpha_{10} - \alpha_{11}i) \\ \beta_2 &= \frac{1}{2}(\alpha_{00} - \alpha_{01} + \alpha_{10} - \alpha_{11}) \\ \beta_3 &= \frac{1}{2}(\alpha_{00} - \alpha_{01}i - \alpha_{10} + \alpha_{11}i) \end{aligned} \quad (24)$$

On peut alors définir la matrice correspondant à l'opérateur **QFT** comme suit:

$$QFT_4 = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix} \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \quad (25)$$

L'algorithme de Shor avec une utilisation d'une **QFT** pour trouver la période a été démontrée expérimentalement en 2001 par **IBM** qui a pu factoriser $P = 15$ en $Q = 3$ et $R = 5$ avec un ordinateur quantique à 6 qubits. Il a été démontré qu'il faudrait en pratique (en l'état actuel des connaissances) environ 10^6 qubits pour factoriser en nombres premiers une clef publique issue de RSA-2048. Actuellement en mai 2024, le plus grand nombre de qubits stables est de 504, record détenu par la société chinoise QuantumCTek Co., Ltd.